

Does Target Class Matter? A Reproducibility Study of Plausible Counterfactual Explanations for Image Classification

Anonymous Submission

Appendix

A Experimental Details

A.1 Network architectures

MLP Relu_4_1024

This network was the only network that we did not train. It was sourced from a neural network verification tool, Eran.¹ The original format was a PyTorch model. This was then translated using the ONNX file format into a keras-tensorflow model. The specifics of the neural architecture can be found in Table 1.

Layer	Details
Input	$1 \times 28 \times 28$, flattened to 1×784
Hidden Layer 1	Fully Connected, 1024 units
ReLU	
Hidden Layer 2	Fully Connected, 1024 units
ReLU	
Hidden Layer 3	Fully Connected, 1024 units
ReLU	
Output Layer	Fully Connected, 10 units

Table 1: Neural network architecture for `mnist_relu_4_1024`, also named `mnist_output_100` in the results.

mnist_cnn

This network was used by Kenny and Keane [2021]. We chose to use the exact same specifications as the original. The only change is that we reduced the number of epochs from 3 times 10 epochs at different learning rates, to just 10 epochs. We use a learning rate of 0.001 and a batch size of 8.

mnist_alibi_cnn

This network was used in the notebook showcasing the method by Van Looveren and Klaise [2021] implemented in the alibi package.² The hyperparameters used were the same as in the alibi notebook. We used 3 epochs, a batch size of 32 and a learning rate of 0.001.

¹<https://github.com/eth-sri/eran>

²https://docs.seldon.io/projects/alibi/en/stable/examples/cfproto_mnist.html

Layer	Details
Conv2d	8 filters, 5×5 , stride 1×1 , padding = 2
Dropout2d	$p = 0.1$
BatchNorm2d	
ReLU	
Conv2d	16 filters, 5×5 , stride 2×2 , padding = 2
Dropout2d	$p = 0.1$
BatchNorm2d	
ReLU	
Conv2d	32 filters, 5×5 , stride 1×1 , padding = 2
Dropout2d	$p = 0.1$
BatchNorm2d	
ReLU	
Conv2d	64 filters, 5×5 , stride 2×2 , padding = 2
Dropout2d	$p = 0.2$
BatchNorm2d	
ReLU	
Conv2d	128 filters, 3×3 , stride 1×1 , padding = 1
GAP	Global Average Pooling
Linear	
SoftMax	128 input units, 10 output units

Table 2: Summary of the CNN architecture in [Kenny and Keane, 2021], also referred to in the results as `mnist_cnn_output_100`.

LeNet5

The LeNet5 network [LeCun *et al.*, 1998] was used. The number of epochs was 10, the batch size 8 and the learning rate 0.001.

ResNet8

We chose to use a shallow ResNet architecture as an alternative to the other networks because larger ResNets would have been too much for MNIST. The hyperparameters that we used were 10 epochs, a batch size of 8 and a learning rate of 0.001.

We used the same neural architecture for CIFAR and changed the channels to work for RGB data. However, with the number of epochs increasing to 30, a batch size of 32 and a learning rate of 0.001.

MLP

This network is based on the network used by Schut *et al.* [2021]. We do not use an ensemble as in the original pa-

Layer	Details
Input	$1 \times 28 \times 28$ image
Conv2d	$1 \rightarrow 64$, kernel 2×2 , padding = 1
ReLU	
MaxPool2d	kernel 2×2 , stride 2×2
Dropout	$p = 0.3$
Flatten	$64 \times 14 \times 14 \rightarrow 12544$
Fully Connected (fc1)	$12544 \rightarrow 256$
ReLU	
Dropout	$p = 0.5$
Fully Connected (fc2)	$256 \rightarrow 10$

Table 3: Summary of the `alibi_CNN` architecture.

Layer	Details
Input	$3 \times 32 \times 32$
Convolution Layer 1	6 filters, 5×5 kernel, stride = 1
ReLU	
Max Pooling Layer 1	2×2 kernel, stride = 2
Convolution Layer 2	16 filters, 5×5 kernel, stride = 1
ReLU	
Max Pooling Layer 2	2×2 kernel, stride = 2
Fully Connected Layer 1	120 units
ReLU	
Fully Connected Layer 2	84 units
ReLU	
Output Layer	Fully Connected, 10 units

Table 5: Summary of architecture for `cifar_captum_cnn`.

per. We use a batch size of 8, a learning rate of 0.001 and 20 epochs.

Layer	Details
Input	$1 \times 28 \times 28$, flattened to 1×784
Hidden Layer 1	Fully Connected, 80 units
BatchNorm	
ReLU	
Hidden Layer 2	Fully Connected, 80 units
BatchNorm	
ReLU	
Output Layer	Fully Connected, 10 units

Table 4: Summary of architecture for MLP, also known in the results file as `mnist_schut_mlp`.

alibi-CF

We used the documentation of alibi⁵ and set the following hyperparameters:

- `gamma` = 100
- `theta` = 100
- `c_init` = 1
- `c_steps` = 2
- `k` = 5
- `max_iterations` = 1000.

alibi-Proto-CF

We used the documentation of alibi (see footnote 5) and set the following hyperparameters:

- `target_proba` = 1.0
- `tol` = 0.1
- `max_iter` = 1000
- `lam_init` = $1e-1$
- `max_lam_steps` = 10
- `learning_rate_init` = 0.1.

A.3 Evaluation metric specifications

Specifically for the IM1 and IM2 metrics, we used the same autoencoder architecture as implemented in [Kenny and Keane, 2021], for both MNIST and CIFAR. For the Moran’s I measure, we use what is known as a queen kernel. That is, every neighbouring pixel (left, right, diagonal) is processed.

A.4 Hardware specifications

To generate all explanations we used an Intel Xeon Gold 5118 CPU with 48 logical processors and a base frequency of 2.30 GHz. All other operations were conducted on a Lenovo Yoga Slim 7 Pro; Processor AMD Ryzen 7 5800H; Memory 16 GB DDR4 3200 MHz; Storage 512 GB SSD; AMD Radeon Graphics (CPU used only).

⁵https://docs.seldon.io/projects/alibi/en/stable/examples/cf_mnist.html

³https://github.com/pytorch/captum/blob/09aa0489b19b09f5fa9238dffedd1a79d277b620/tutorials/CIFAR_Captum_Robustness.ipynb#L542

⁴https://github.com/huyvnphan/PyTorch_CIFAR10/blob/master/cifar10_models/resnet.py

B Additional Visualisations and Experiments

In this section, we present additional visualisations and experiments that complement and reinforce the findings discussed in the main body of the paper. Specifically, Figures 2 to 10 illustrate results analogous to those shown in Figure 3 of the main paper, but extended across all evaluated methods and for both the MNIST and CIFAR datasets.

B.1 To what extent does the performance of different counterfactual explanation methods change for different neural architectures?

While it may seem intuitive, the performance of a counterfactual explanation method can vary significantly depending on the network architecture employed. In our analysis, we observed substantial variation between networks. For instance, when comparing the MLP Relu_4_1024 to the CNN utilised in [Kenny and Keane, 2021], alibi-Proto-CF achieves near-perfect performance on the former, but its success rate drops to 27.7% on the latter. To further illustrate this across all networks, Figure 1 presents an overview of all evaluated networks for both datasets, ranking methods on success and weak success metrics. The results highlight a clear distinction between different networks: whilst alibi-Proto-CF ranks highest on the networks MLP Relu_4_1024 and LenNet5, it also ranks worst on ResNet8 and the CNN tested in [Kenny and Keane, 2021]. In particular, we observe that for the success metric, the relative ranking of methods can change substantially depending on the network. This indicates that a method that performs well on one network may perform relatively poorly on another. This effect is less pronounced when evaluated with the weak success metric. Notably, alibi-Proto-CF consistently identifies at least one explanation across all networks. For CIFAR, we again observe evidence of reordering under the success metric, though to a lesser extent than in MNIST, while performance under the weak success metric remains largely unchanged across methods and networks.

B.2 Additional Visualizations

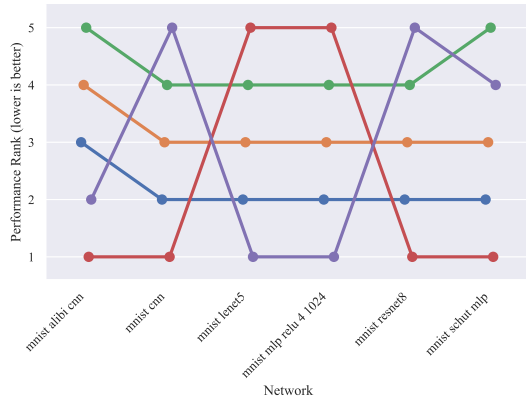
Across nearly all methods and datasets, a consistent improvement in the average success rate is observed once timeout cases are excluded. This pattern provides additional evidence that timeouts contribute significantly to failures in generating valid counterfactual explanations, particularly in regions of the input space where decision boundaries are more complex or computationally demanding to traverse. The main exceptions to this trend appear in Figures 7 and 8, corresponding to the Min-Edit and C-Min-Edit methods, respectively, where the removal of timeouts results in negligible changes. This stability could be attributed to the already high baseline success rates of these methods, with their lowest observed average values being 0.89 for Min-Edit and 0.94 for C-Min-Edit.

A further noteworthy pattern emerges in Figures 5 and 10, which depict results for the alibi-Proto-CF method. These heatmaps reveal distinct vertical structures, indicating that certain target classes consistently yield higher success rates regardless of the original predicted class. This suggests that some target classes are inherently easier to reach or better represented within the prototype space, highlighting poten-

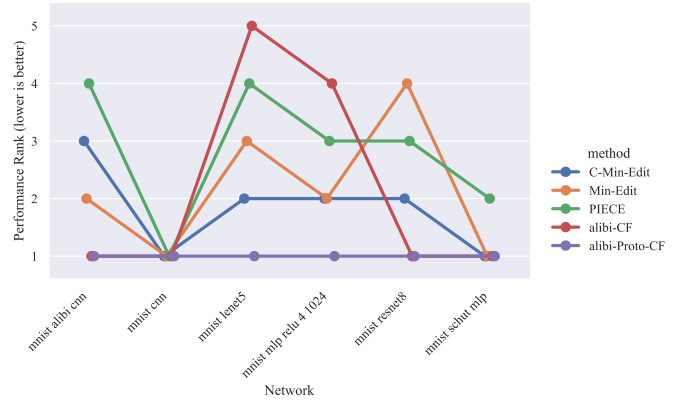
tial asymmetries in how counterfactual explanations are distributed across the target domain.

References

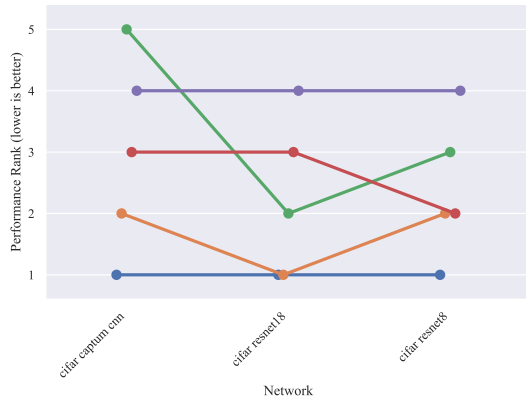
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- Eoin M. Kenny and Mark T. Keane. On generating plausible counterfactual and semi-factual explanations for deep learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 11575–11585, 2021.
- Narine Kokhlikyan, Vivek Miglani, Miguel Martin, Edward Wang, Bilal Alsallakh, Jonathan Reynolds, Alexander Melnikov, Natalia Kliushkina, Carlos Araya, Siqi Yan, and Orion Reblitz-Richardson. Captum: A unified and generic model interpretability library for PyTorch. *arXiv preprint arXiv:2009.07896*, 2020.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- Lisa Schut, Oscar Key, Rory Mc Grath, Luca Costabello, Bogdan Sacaleanu, Medb Corcoran, and Yarin Gal. Generating interpretable counterfactual explanations by implicit minimisation of epistemic and aleatoric uncertainties. In *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics*, pages 1756–1764, 2021.
- Arnaud Van Looveren and Janis Klaise. Interpretable counterfactual explanations guided by prototypes. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 650–665. Springer, 2021.



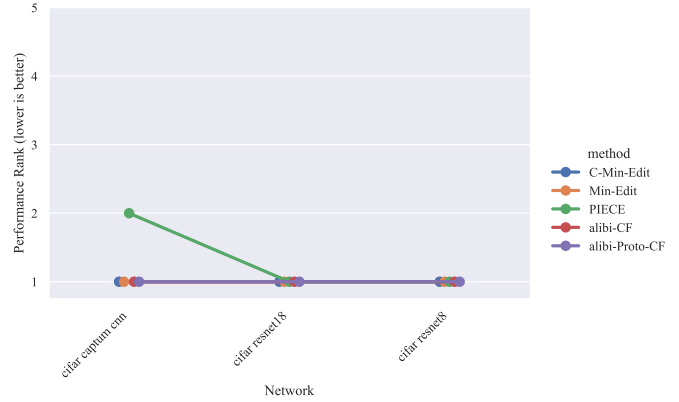
(a) Success on MNIST



(b) Weak success on MNIST



(c) Success on CIFAR



(d) Weak success on CIFAR

Figure 1: Performance Rank of the average success and weak-success rates for each counterfactual explanation method across all MNIST networks. 'Success' and 'weak success' are binary indicators denoting whether a generated counterfactual explanation fully or partially achieves the target class, respectively. For each network, the average success rate per method is computed, and methods are ranked accordingly; lower ranks indicate better performance.

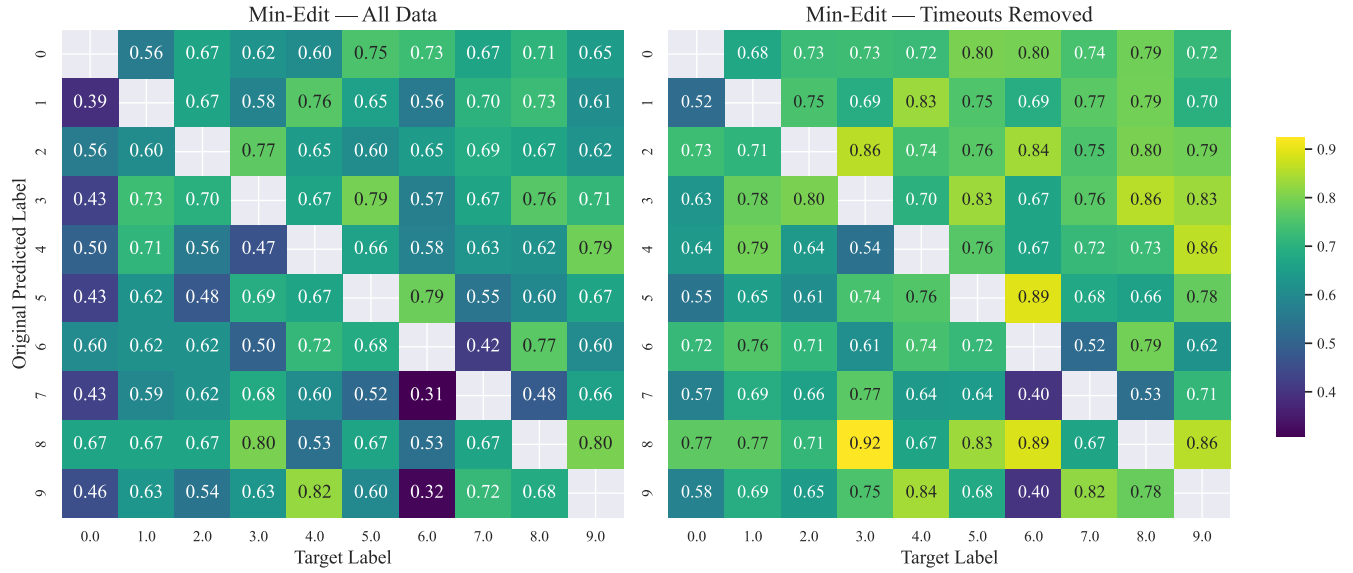


Figure 2: Average Success for each original class and target class pair for the Min-Edit method on MNIST. Timeouts are either included in the average success calculation or filtered out.

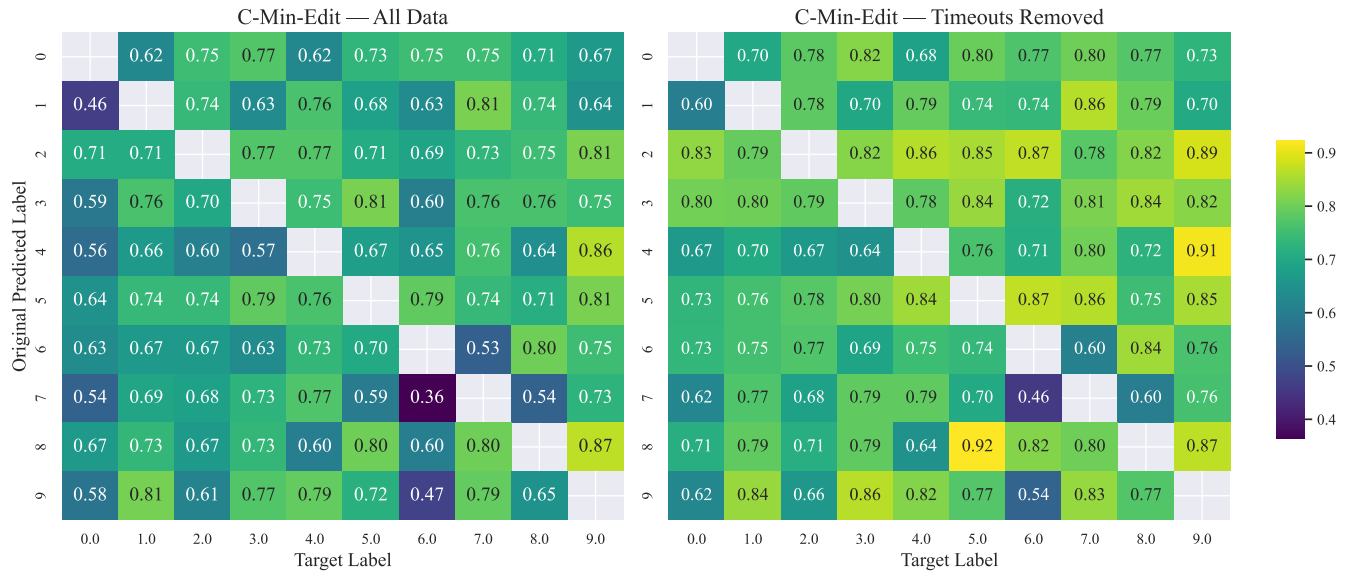


Figure 3: Average Success for each original class and target class pair for the C-Min-Edit method on MNIST. Timeouts are either included in the average success calculation or filtered out.

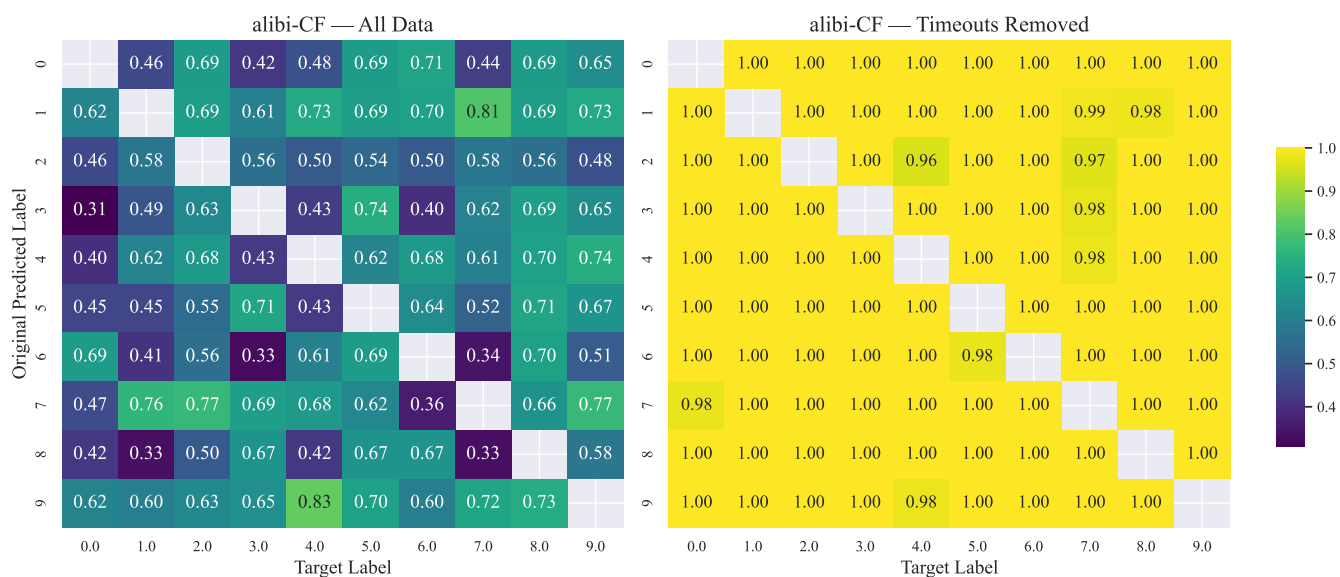


Figure 4: Average Success for each original class and target class pair for the alibi-CF method on MNIST. Timeouts are either included in the average success calculation or filtered out.

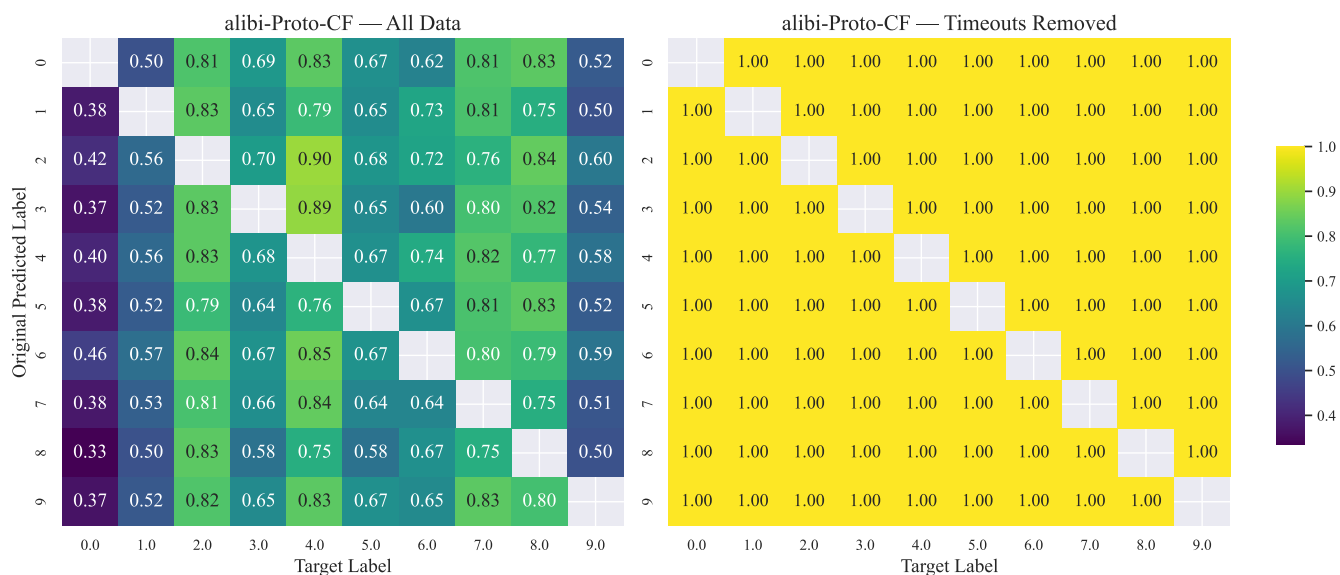


Figure 5: Average Success for each original class and target class pair for the alibi-Proto-CF method on MNIST. Timeouts are either included in the average success calculation or filtered out.

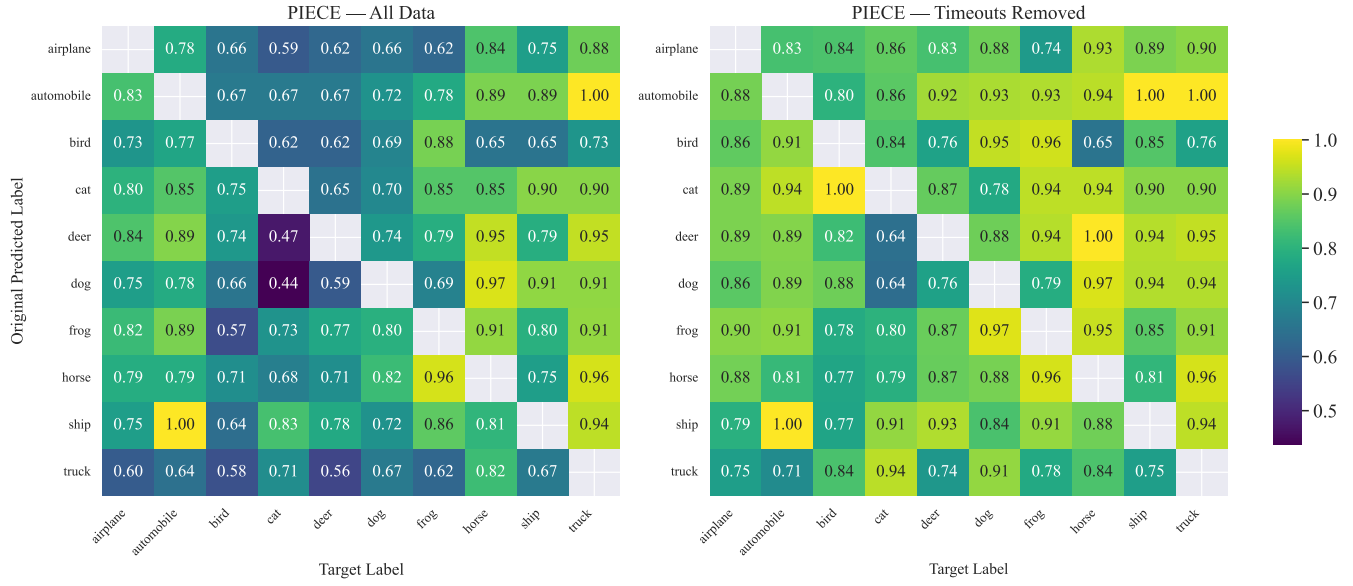


Figure 6: Average Success for each original class and target class pair for the PIECE method on CIFAR. Timeouts are either included in the average success calculation or filtered out.



Figure 7: Average Success for each original class and target class pair for the Min-Edit method on CIFAR. Timeouts are either included in the average success calculation or filtered out.

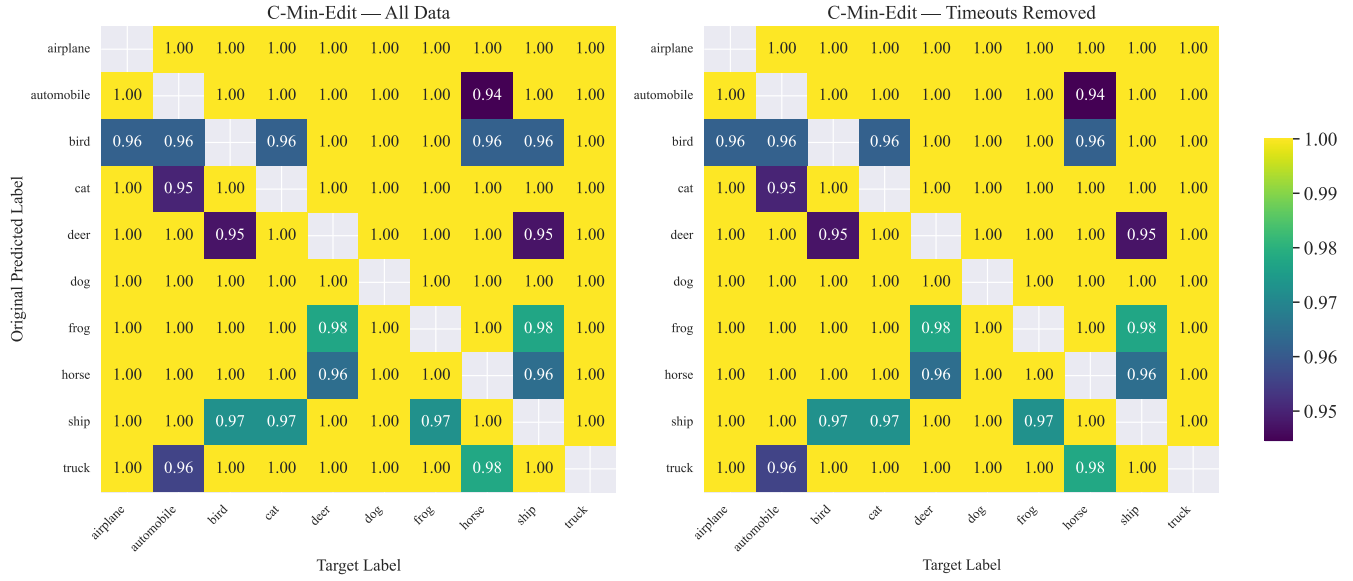


Figure 8: Average Success for each original class and target class pair for the C-Min-Edit method on CIFAR. Timeouts are either included in the average success calculation or filtered out.



Figure 9: Average Success for each original class and target class pair for the alibi-CF method on CIFAR. Timeouts are either included in the average success calculation or filtered out.

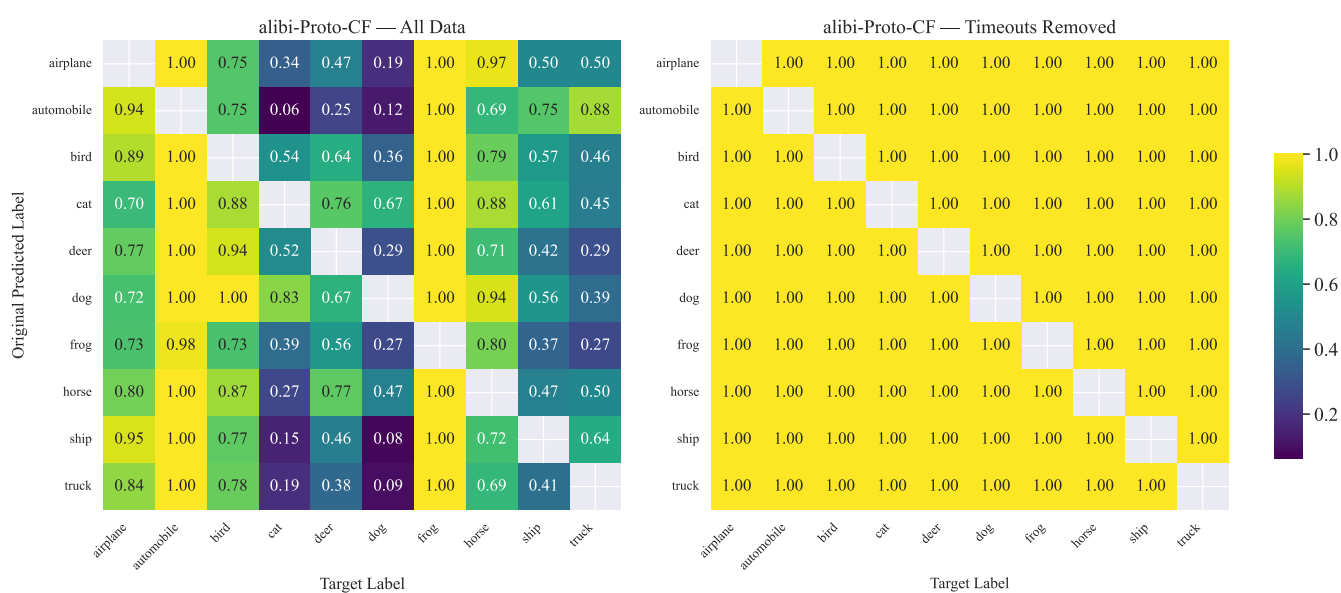


Figure 10: Average Success for each original class and target class pair for the alibi-Proto-CF method on CIFAR. Timeouts are either included in the average success calculation or filtered out.